

Relazione del Progetto

“CluedoLite”

Abbondanza Giulia

Pivi Giorgia

Raimondi Gessica

Severi Valentina

10 giugno 2026

Indice

1 Analisi

1.1 Descrizione e requisiti 3

1.2 Modello del Dominio 4

2 Design

2.1 Architettura 2

2.2 Design dettagliato 9

2.2.1 Giulia Abbondanza 8

2.2.2 Giorgia Pivi 13

2.2.3 Gessica Raimondi 16

2.2.4 Valentina Severi 16

3 Sviluppo

3.1 Testing automatizzato 19

3.2 Note di sviluppo 20

3.2.1 Giulia Abbondanza 8

3.2.2 Giorgia Pivi 13

3.2.3 Gessica Raimondi 16

3.2.4 Valentina Severi 16

4 Commenti finali

4.1 Autovalutazione e lavori futuri

4.1.1 Giulia Abbondanza 8

4.1.2 Giorgia Pivi	13
4.1.3 Gessica Raimondi	16
4.1.4 Valentina Severi	16
4.2 Difficoltà incontrate e commenti per i docenti	24
4.2.1 Giulia Abbondanza	8
4.2.2 Giorgia Pivi	13
4.2.3 Gessica Raimondi	16
4.2.4 Valentina Severi	16
A Guida utente	26

1.1 Analisi

1.1 Descrizione e requisiti

L'applicazione CluedoLite è una versione digitale semplificata del celebre gioco investigativo Cluedo.

Il suo obiettivo è permettere agli utenti di vivere un'esperienza di gioco completa, mantenendo le principali dinamiche di deduzione logica del gioco originale, ma con regole alleggerite e un'interfaccia grafica intuitiva.

L'applicazione è rivolta a utenti interessati ai giochi di logica e investigazione e consente di svolgere una partita completa in formato digitale, automatizzando la gestione delle regole, delle carte e dei turni di gioco.

Il software gestisce l'intero flusso di una partita: dalla creazione della soluzione segreta alla distribuzione delle carte, dal movimento dei personaggi sulla mappa alla formulazione di sospetti e accuse, fino alla determinazione automatica della vittoria o della sconfitta.

Requisiti funzionali

I requisiti funzionali includono:

- Gestione dei giocatori: scelta del numero di partecipanti e gestione dei turni di gioco.
- Creazione della soluzione segreta: selezione casuale di un sospettato, un'arma e una stanza all'inizio della partita.
- Distribuzione delle carte: assegnazione automatica delle carte rimanenti ai giocatori.
- Rappresentazione grafica del tabellone: visualizzazione di un tabellone bidimensionale contenente le stanze del gioco.
- Movimento dei personaggi: spostamento tra stanze adiacenti secondo regole semplificate.
- Formulazione dei sospetti: possibile solo quando il giocatore si trova in una stanza.
- Verifica dei sospetti: il sistema mostra automaticamente una carta che confuta l'ipotesi, se presente.
- Accusa finale: possibilità di formulare un'accusa e gestione dell'esito (vittoria/sconfitta).
- Gestione delle condizioni di fine partita: determinazione automatica del vincitore.

Requisiti funzionali opzionali

- Tabella dei sospetti per ogni giocatore, aggiornata automaticamente.
- Grafica delle carte (armi, personaggi, stanze).
- Scelta del personaggio da parte del giocatore.

Requisiti non funzionali

- Interfaccia grafica semplice, chiara e coerente.
- Prevenzione di azioni illegali in base allo stato del gioco (es. effettuare due sospetti in un turno) e gestione corretta delle regole.
- Mantenere un equilibrio tra fedeltà al gioco originale e riduzione della complessità.

1.2 Modello del Dominio

La **Partita** deve avere accesso a un **Gestore dei Turni**, responsabile dell'avanzamento del gioco e dell'applicazione delle regole fondamentali di Cluedo Lite. La partita deve inoltre conoscere il **Tabellone**, composto da un insieme di **Stanze**, così da avere un riferimento concreto per la posizione dei giocatori e per la formulazione dei sospetti.

Il **Gestore dei Turni** è a conoscenza delle regole di gioco e ne impone il rispetto durante l'intera partita. In base allo stato corrente della partita, il Gestore dei Turni è in grado di associare il comportamento corretto per gestire il turno del giocatore attivo: movimento (le stanze devono poter essere collegate tra loro tramite relazioni di adiacenza, così da definire percorsi validi per gli spostamenti), possibilità di formulare un sospetto, eventuale accusa finale e fine turno.

Ogni **Giocatore** controlla un **Personaggio** e possiede un insieme di **Carte**, distribuite all'inizio della partita. Le carte appartengono a tre categorie (personaggi, armi, stanze) e una combinazione di esse costituisce la **Soluzione Segreta**, estratta casualmente all'avvio della partita.

Il giocatore può formulare un **Sospetto** quando si trova all'interno di una stanza. Il sospetto è composto da un personaggio, un'arma e la stanza corrente. Gli altri giocatori, secondo l'ordine stabilito, possono mostrare una carta che confuta il sospetto.

L'**Accusa** rappresenta invece la dichiarazione finale del giocatore: se la combinazione

scelta coincide con la soluzione segreta, la partita termina con la sua vittoria; in caso contrario, il giocatore viene eliminato dal gioco.

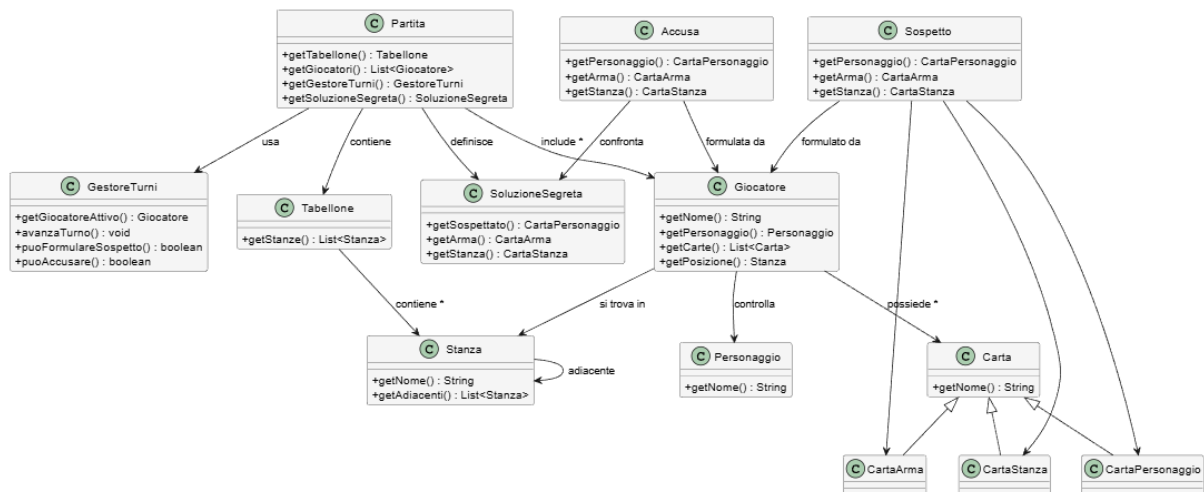


Figura 1.1: Schema UML dell'analisi del dominio, con rappresentate le entità principali ed i rapporti fra loro

2. Design

2.1 Architettura

L'architettura di CluedoLite segue il pattern architetturale **MVC**. Il controller principale del pacchetto **menucontroller** e il **GameController**, svolgono il ruolo di entry point dell'applicazione, in quanto gestiscono sia il passaggio tra le diverse schermate della view sia le operazioni preliminari necessarie alla creazione della partita, come l'inizializzazione dei giocatori, la generazione delle carte e la preparazione del tabellone.

Il **GameBoard** e il **TurnManager** presenti nel model costituiscono il nucleo logico del gioco: essi coordinano l'avanzamento dei turni, stabiliscono quali azioni sono consentite al giocatore attivo e applicano le regole fondamentali del gioco. Il controller che interagisce maggiormente con il model è quello del pacchetto **gameboardcontroller**, che si occupa di interpretare le azioni dell'utente sulla plancia e di tradurle in operazioni sul modello, come il movimento del personaggio.

L'entry point della view è invece rappresentato da **menuview**, che permette di aprire **GameView** che racchiude i vari componenti grafici contenuti nei pacchetti **gameboardview**, **suspicionview**, **accuseview**, **tableview**, **buttonflowview**, **endturnbuttonview**. Questi componenti definiscono i metodi necessari per cambiare schermata e aggiornare la GUI in base allo stato del gioco.

La sincronizzazione tra Model e View avviene interamente tramite i Controller, che

leggono lo stato del Model e aggiornano la View in modo esplicito. Questo approccio mantiene il Model indipendente dalla GUI e garantisce un forte disaccoppiamento: per sostituire la View è sufficiente reimplementare i componenti grafici e collegarli ai Controller esistenti, senza modificare la logica del Model.

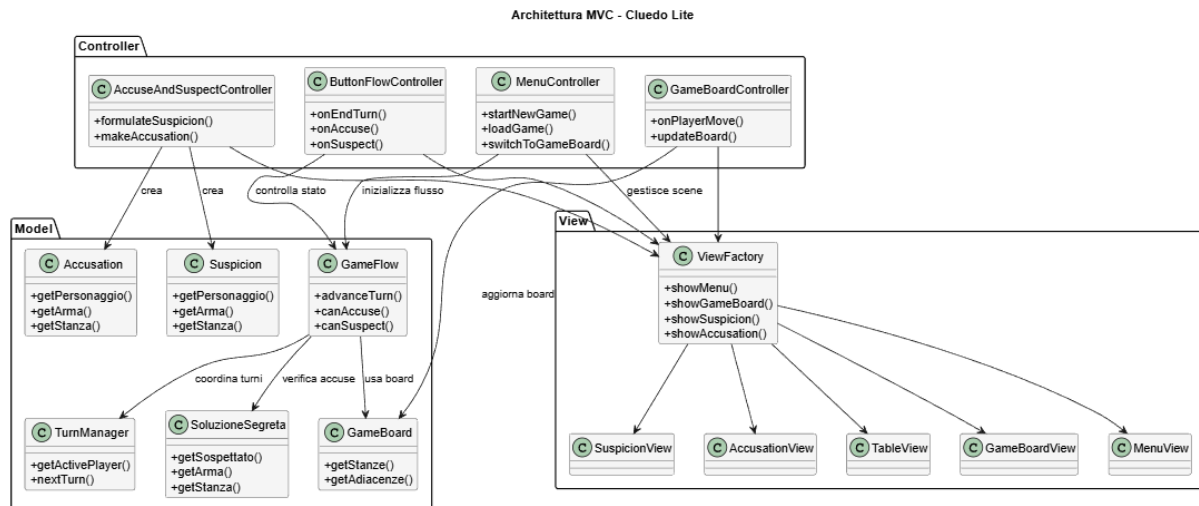
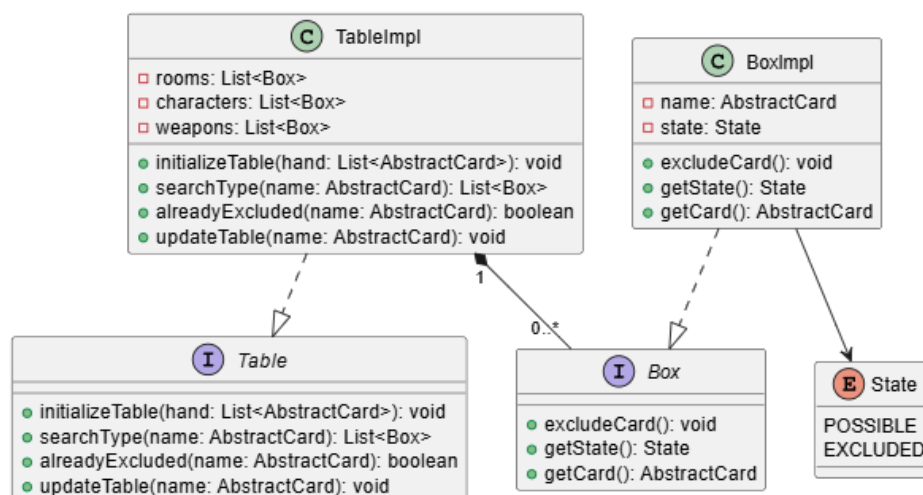


Figura 2.1: Schema UML dell'architettura, con rappresentati i principali attori del pattern MVC

2.2 Design dettagliato

2.2.1 Giulia Abbondanza

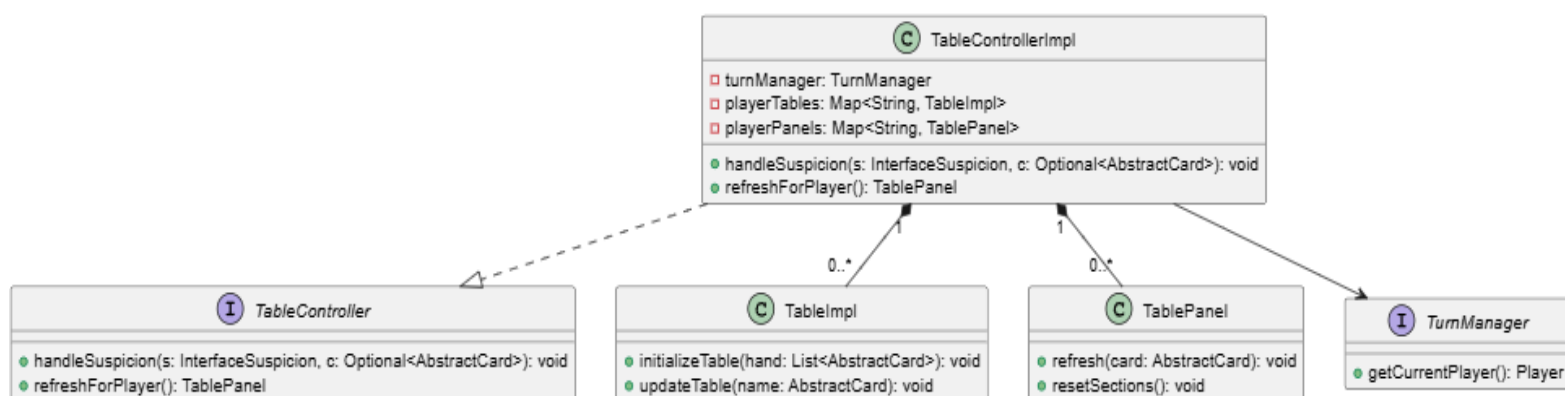
Modellazione della tabella dei sospetti



Problema Durante una partita, ciascun giocatore ha necessità di tenere traccia di quali carte siano già state escluse dalla soluzione e quali rimangano ancora possibili. La tabella deve essere inizializzata automaticamente escludendo le carte già in mano al giocatore, aggiornarsi ogni volta che un avversario mostra una carta durante la verifica di un sospetto, e restituire in modo sicuro le informazioni senza esporre la struttura interna.

Soluzione Si è introdotta una gerarchia composta dall'interfaccia Box, implementata da BoxImpl, che associa a ogni carta un'istanza dell'enumerazione State (POSSIBLE / EXCLUDED). La classe TableImpl mantiene tre liste interne distinte per personaggi, armi e stanze, popolate all'inizializzazione tramite Deck.getAllCards(). Il metodo initializeTable() esclude automaticamente le carte già in possesso del giocatore, mentre updateTable() esclude una carta specifica quando viene mostrata durante la verifica di un sospetto. Il metodo searchType() restituisce una copia immutabile tramite List.copyOf(), garantendo che il chiamante non possa alterare lo stato interno del model.

Gestione per-giocatore della tabella tramite lazy initialization

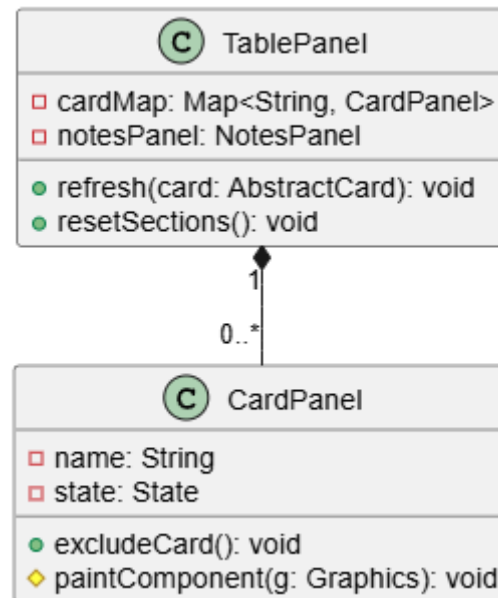


Problema In una partita multi-giocatore, ogni giocatore possiede la propria tabella dei sospetti indipendente, inizializzata con le sue carte specifiche. Il controller deve poter recuperare la tabella e il pannello grafico corretti al cambio di turno, senza crearli in anticipo per tutti i giocatori.

Soluzione Il TableControllerImpl mantiene due HashMap<String, ...> che associano il nome del giocatore rispettivamente a TableImpl e TablePanel. Il metodo refreshForPlayer() utilizza computeIfAbsent() per creare la tabella e il pannello solo la prima volta che il giocatore diventa attivo. Questo approccio evita allocazioni inutili

all'avvio e garantisce che lo stato di ogni giocatore sia isolato e persistente per l'intera durata della partita.

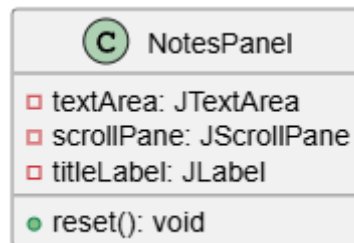
Visualizzazione grafica della tabella con sezioni collassabili



Problema La tabella dei sospetti contiene 21 carte suddivise in tre categorie. Mostrare tutte le carte contemporaneamente occuperebbe eccessivo spazio nell'interfaccia, rendendo difficile la lettura. È necessario che il giocatore possa espandere e collassare le sezioni a piacimento, e che l'aspetto grafico di ogni carta rifletta in tempo reale il suo stato (possibile o esclusa).

Soluzione Il **TablePanel** è organizzato in tre sezioni collassabili, ciascuna composta da un **JLabel** cliccabile che funge da intestazione e da un **JPanel** di **CardPanel** sottostante, visibile o nascosto tramite `setVisible()`. Ogni **CardPanel** ridefinisce `paintComponent()` per disegnare l'immagine della carta: se lo stato è **EXCLUDED**, sovrappone una tinta scura semitrasparente e disegna una X rossa con bordo, rendendo immediatamente riconoscibili le carte escluse. Il metodo `refresh()` di **TablePanel** aggiorna il solo **CardPanel** interessato, evitando ridisegni inutili dell'intera vista.

Blocco note personale collassabile

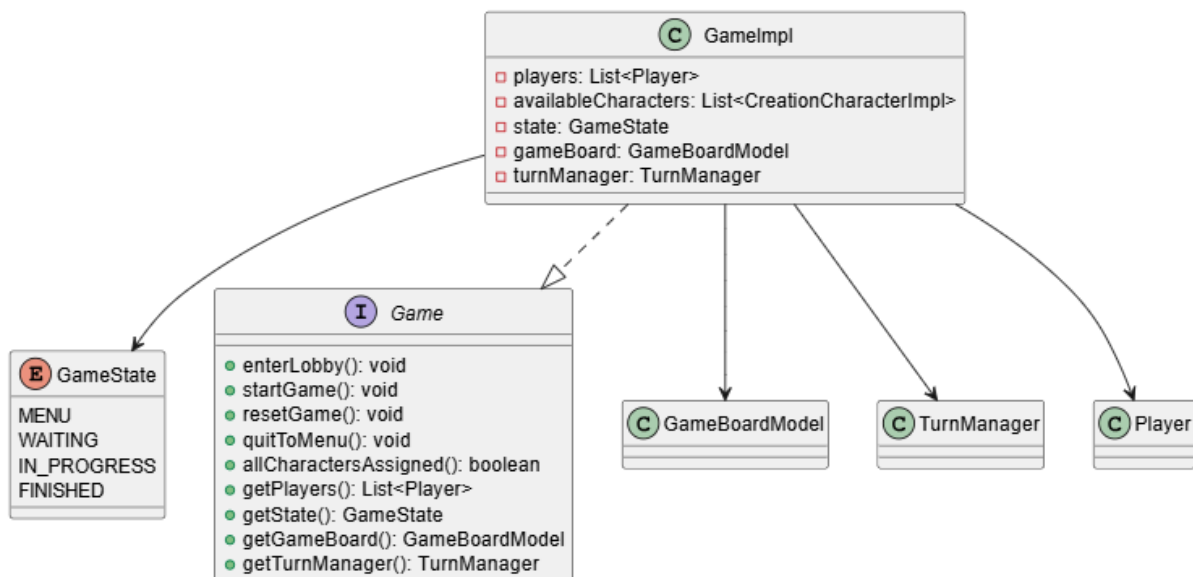


Problema Nel gioco da tavolo originale i giocatori prendono appunti su un foglio fisico. È necessario offrire un'area testuale digitale equivalente, che non occupi spazio fisso nell'interfaccia quando non utilizzata.

Soluzione Il NotesPanel implementa un blocco note collassabile: cliccando il titolo "Notes", un JScrollPane contenente una JTextArea diventa visibile o si nasconde. La JTextArea mostra un testo placeholder ("Click to write...") quando vuota, che scompare al primo clic tramite un MouseListener, e ricompare se il focus viene perso con il campo ancora vuoto, gestito da un FocusListener. Il metodo reset() riporta il pannello allo stato collassato al cambio di turno, garantendo la privacy degli appunti di ciascun giocatore.

2.2.2 Giorgia Pivi

Gestione dello stato della partita tramite enumerazione

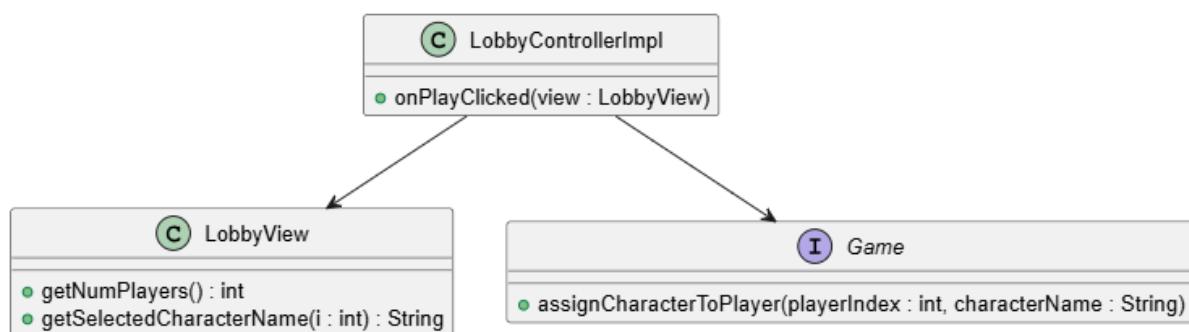


Problema: Il ciclo di vita della partita è caratterizzato da transizioni di stato ben definite: dall'accensione del gioco (menu principale) alla selezione dei personaggi

(lobby), fino all'avvio della partita vera e propria. È necessario impedire azioni illegali come tentare di avviare la partita senza che tutti i giocatori abbiano un personaggio assegnato, o resettare il gioco quando nessuna partita è in corso.

Soluzione: Si è introdotta l'enumerazione *GameState* con quattro stati distinti: *MENU*, *WAITING*, *IN_PROGRESS* e *FINISHED*, che definiscono univocamente il ciclo di vita. La classe *GameImpl* tiene memoria dello stato attuale del gioco e usa metodi che cambiano stato solo quando il passaggio è consentito. Ad esempio, *enterLobby()* è consentito solo se lo stato è *MENU*, mentre *startGame()* richiede che lo stato sia *WAITING* e che tutti i giocatori abbiano un personaggio mediante il controllo *allCharactersAssigned()*. Qualora un passaggio non sia valido, il metodo lancia un'eccezione *IllegalStateException* con un messaggio descrittivo. Questo approccio delega al model la responsabilità di mantenere l'invariante di stato, impedendo che il gioco raggiunga configurazioni incoerenti indipendentemente dalle azioni dell'utente.

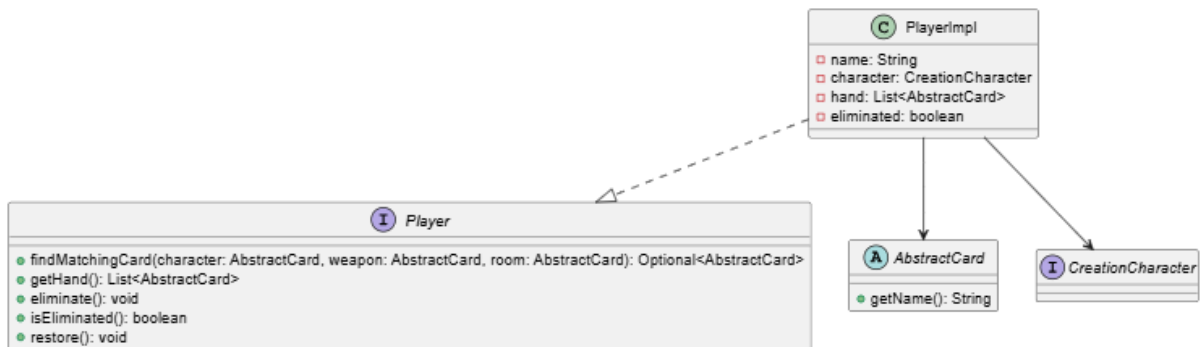
Validazione dell'unicità dei personaggi in fase di lobby



Problema: Nel gioco da tavolo, ogni giocatore può controllare un solo personaggio unico. Quando i giocatori selezionano i loro personaggi dalla lobby, il sistema deve garantire che nessuno due giocatori abbiano scelto lo stesso personaggio prima di permettere l'avvio della partita.

Soluzione: La validazione è implementata nel metodo *onPlayClicked()* di *LobbyControllerImpl* che per ogni coppia di giocatori (i, j) con $i < j$, si confrontano i loro personaggi selezionati. Se due giocatori hanno scelto lo stesso personaggio, la funzione interrompe immediatamente il flusso e mostra un messaggio di errore via *JOptionPane.showMessageDialog()*. Solo se il controllo passa con successo procede alla creazione del modello e all'assegnazione tramite *game.assignCharacterToPlayer()*. Centralizzare questa validazione nel controller anziché delegarla al model mantiene la view isolata dalla logica di business, rispettando il pattern MVC.

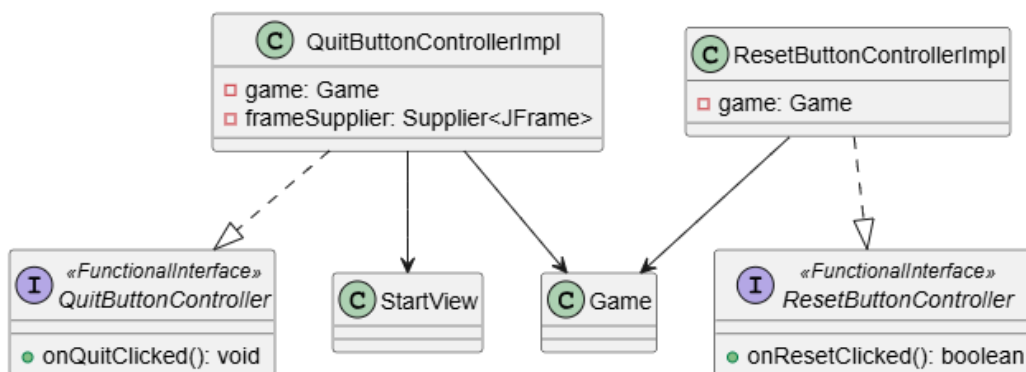
Gestione della scelta del personaggio e dello stato del giocatore



Problema: Ogni giocatore ha un nome, è associato a un personaggio durante la partita, possiede un insieme di carte e può essere eliminato. È necessario modellare il giocatore in modo che il suo stato sia gestibile indipendentemente dalla rappresentazione grafica, e che supporti operazioni di ripristino in caso di reset della partita.

Soluzione: L'interfaccia **Player** definisce il contratto per le operazioni di base su un giocatore: `chooseCharacter(CreationCharacterImpl character)` per assegnare il personaggio, `getCharacter()` per recuperarlo, `addCard()`, `getHand()` per la gestione delle carte, e infine `eliminate()`, `isEliminated()`, `restore()` per la gestione dello stato di eliminazione. La classe **PlayerImpl** realizza questo contratto mantenendo il personaggio come istanza di **CreationCharacter**, le carte in una `ArrayList<Card>`, e uno stato booleano `eliminated`. La classe **CreationCharacterImpl** estende la classe base **Characters** (parte della gerarchia delle carte) aggiungendo il campo `color`. Il metodo `findMatchingCard()` utilizza `Collections.shuffle()` seguito da `Stream.filter().findFirst()`, restituendo un `Optional<AbstractCard>` per rappresentare esplicitamente l'assenza di una carta corrispondente senza ricorrere a `null`. Il metodo `getHand()` restituisce una copia difensiva tramite `List.copyOf()`, impedendo modifiche esterne alla mano del giocatore.

Controller dei pulsanti di flusso e conferma delle azioni critiche

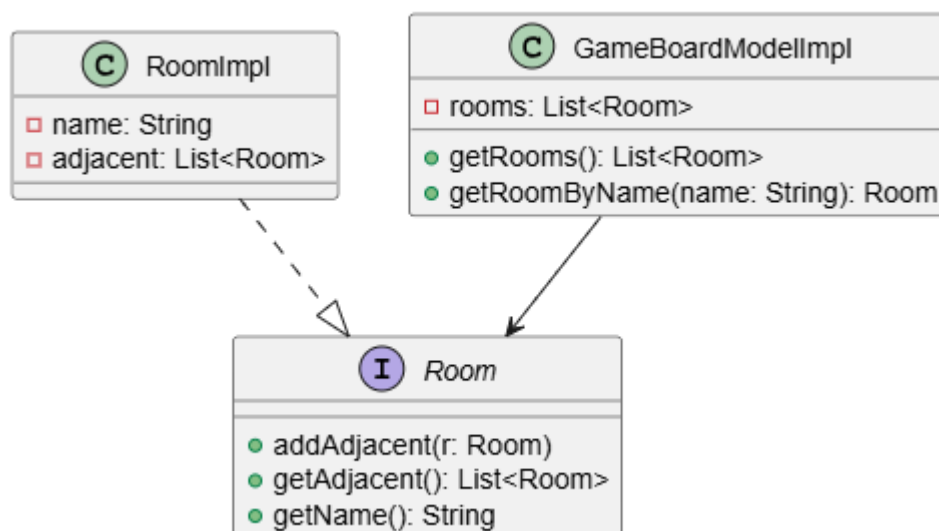


Problema: L'utente deve poter uscire dalla partita (*Quit*) o resettarla (*Reset*) in qualsiasi momento, ma queste azioni modificano lo stato globale del gioco e non devono essere accidentali. È necessario mostrare una finestra di dialogo di conferma senza accoppiare il controller alla logica di presentazione.

Soluzione: Le interfacce *QuitButtonController* e *ResetButtonController* definiscono rispettivamente i metodi *onQuitClicked()* e *onResetClicked()*, implementati da *QuitButtonControllerImpl* e *ResetButtonControllerImpl*. Entrambi ricevono un'istanza di *Game* nel costruttore, mantenendo un riferimento all'interfaccia anziché all'implementazione concreta. Quando invocati, mostrano una finestra *JOptionPane.showConfirmDialog()* con opzioni YES/NO. Se l'utente conferma, i due controller si comportano in modo distinto: *QuitButtonControllerImpl* invoca *game.quitToMenu()*, che azzerla la lista dei giocatori, ripristina i personaggi disponibili e reimposta board e turn manager. Successivamente apre direttamente una nuova *StartView* e dispone il frame corrente. In questo controller è stato utilizzato un *Supplier<JFrame>* per ottenere il riferimento alla finestra corrente in modo lazy, evitando di catturare il frame prima che sia disponibile. *ResetButtonControllerImpl* invoca in sequenza *game.resetGame()* e *game.startGame()*, riportando la partita allo stato iniziale con gli stessi giocatori. Il metodo restituisce true se l'utente ha confermato, false altrimenti.

2.2.3 Gessica Raimondi

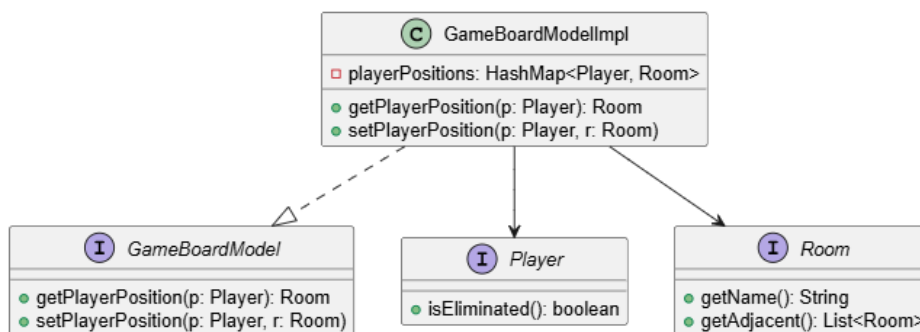
Rappresentazione delle stanze e delle adiacenze



Problema: Il tabellone di Cluedo è composto da nove stanze collegate tra loro da relazioni di adiacenza. È necessario modellare queste relazioni in modo che il sistema possa verificare rapidamente se due stanze sono collegate, senza ricorrere a strutture dati complesse o controlli sparsi nel codice.

Soluzione: Le stanze sono modellate tramite l'interfaccia *Room*, implementata da *RoomImpl*, che mantiene internamente una lista di stanze adiacenti. Poiché le stanze del tabellone di Cluedo sono fisse e disposte sempre nello stesso ordine, si è scelto di organizzarle in una lista circolare: ogni stanza è adiacente alla precedente e alla successiva, e l'ultima è adiacente alla prima. Questo ha permesso di generare tutte le adiacenze automaticamente nel costruttore di *GameBoardModelImpl*, senza doverle definire manualmente una per una. Si è scelto di incapsulare le adiacenze direttamente nella stanza perché in questo modo ciascuna *Room* contiene tutte le informazioni che la riguardano, facilitando l'aggiunta di nuove stanze in futuro senza dover modificare strutture esterne.

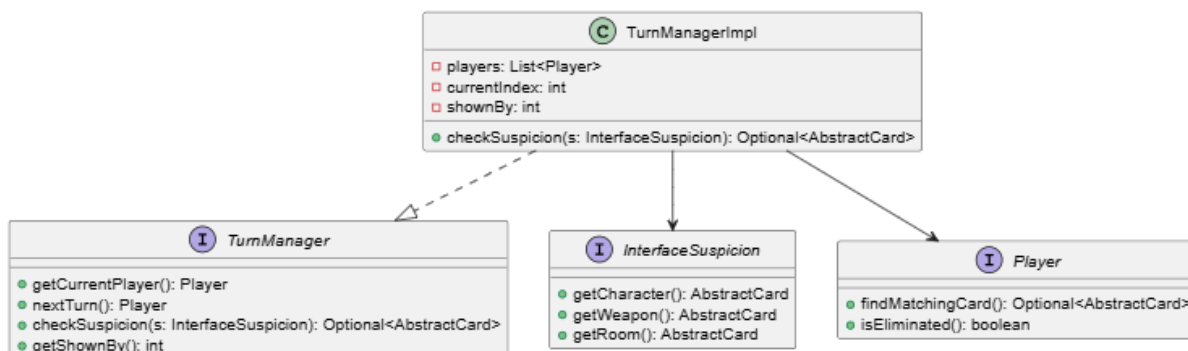
Tracciamento della posizione dei giocatori



Problema Durante la partita il sistema deve sapere in ogni momento in quale stanza si trova ciascun giocatore, aggiornare questa informazione ad ogni spostamento e gestire il caso in cui un giocatore non sia ancora stato posizionato sul tabellone all'inizio della partita.

Soluzione La posizione di ogni giocatore è mantenuta in una *HashMap<Player, Room>* all'interno di *GameBoardModelImpl*. La scelta di tenerla nel model del tabellone e non nel giocatore stesso è motivata dal fatto che il giocatore non deve conoscere il tabellone: è *GameBoardModelImpl* il componente responsabile di sapere dove si trovano i giocatori durante la partita. Il valore *null* restituito quando un giocatore non è ancora presente nella mappa viene sfruttato esplicitamente dal controller per distinguere il primo posizionamento dai turni successivi, evitando la necessità di introdurre uno stato aggiuntivo nel model.

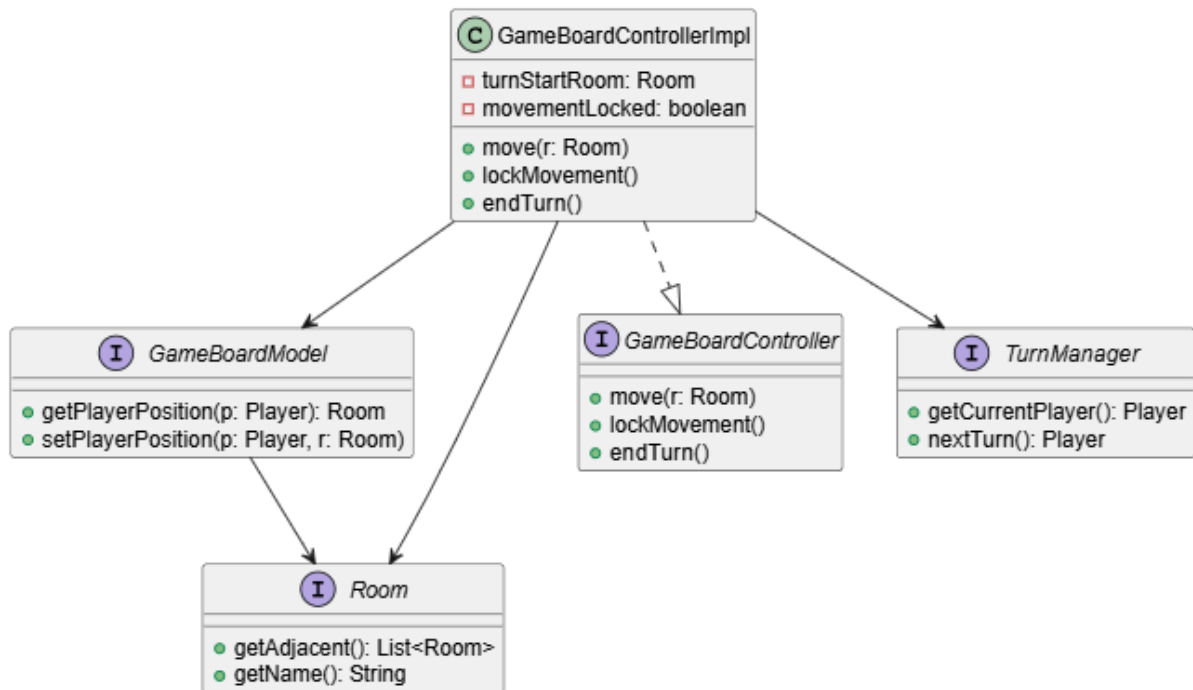
Risoluzione circolare dei sospetti



Problema: Quando un giocatore formula un sospetto, gli altri giocatori devono essere interrogati in ordine circolare a partire dal giocatore successivo. Il primo in grado di mostrare una carta che confuta il sospetto deve farlo e la ricerca deve fermarsi. Se nessun giocatore può confutare il sospetto, il sistema deve comunicarlo esplicitamente senza ricorrere a valori *null*.

Soluzione: La logica di risoluzione del sospetto è stata collocata nel *TurnManager* perché è il componente del model che già conosce la lista dei giocatori e l'indice del giocatore corrente. Inoltre, concentrare questa logica in un unico posto significa che eventuali modifiche future alle regole di risoluzione del sospetto, come saltare i giocatori eliminati o cambiare l'ordine di interrogazione, richiederebbero modifiche solo al *TurnManager* senza toccare il resto del sistema. La ricerca parte dal giocatore successivo a quello che ha formulato il sospetto e prosegue in ordine circolare, fermandosi al primo giocatore che possiede una carta corrispondente. Si è scelto di restituire un *Optional<AbstractCard>* perché, a differenza di *null*, costringe esplicitamente il chiamante a gestire il caso in cui nessun giocatore possa rispondere: per accedere al valore è necessario chiamare metodi come *isPresent()* o *ifPresent()*, rendendo impossibile ignorare accidentalmente il caso di assenza e ottenere un errore a runtime.

Separazione tra regole generali di adiacenza e regole del turno

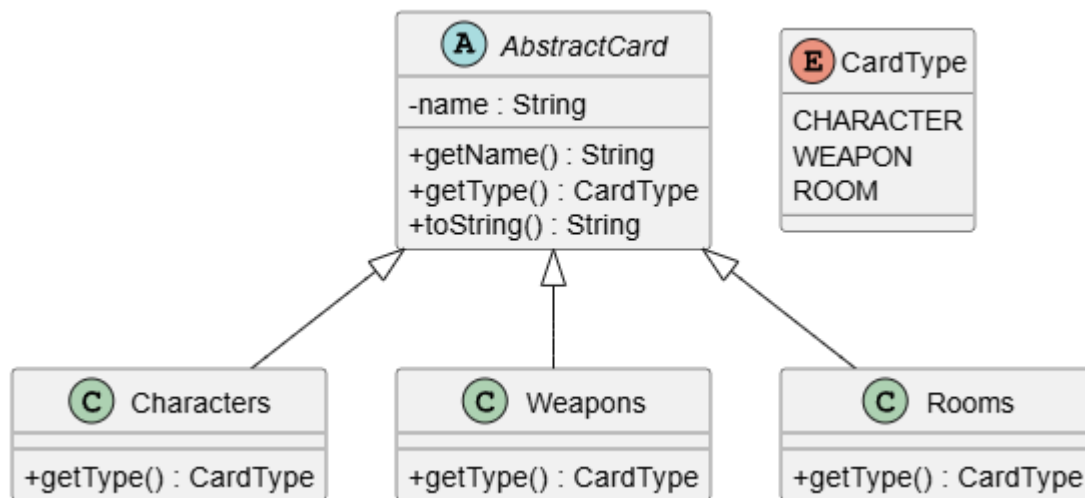


Problema Il model deve esporre regole generali di movimento valide in qualsiasi contesto, mentre il controller deve applicare regole più restrittive legate al flusso del turno. Mescolare queste due responsabilità nello stesso componente avrebbe reso il model dipendente dalla logica di gioco, rendendolo più difficile da testare e riutilizzare.

Soluzione Il *GameBoardModelImpl* si occupa della struttura del tabellone: conosce le stanze e la posizione corrente di ogni giocatore, mentre le adiacenze sono responsabilità di ciascuna *RoomImpl*. Il *GameBoardControllerImpl* introduce una regola legata al turno tramite *turnStartRoom*. Il controller distingue due casi: al primo turno il giocatore non ha ancora una posizione e può scegliere liberamente qualsiasi stanza, che viene salvata come *turnStartRoom*; nei turni successivi sarà la stanza in cui il giocatore si trova già all'inizio del turno. In entrambi i casi, una volta impostata, il giocatore può rimanere lì oppure spostarsi in una delle sue due stanze adiacenti. Poiché *turnStartRoom* è un singolo campo e non viene salvata per ogni giocatore, viene reimpostata a *null* in *endTurn()* ad ogni cambio di turno. Separare queste due responsabilità garantisce che il *GameBoardModel* rimanga focalizzato sulla struttura del tabellone, senza dover conoscere il concetto di turno.

2.2.4 Valentina Severi

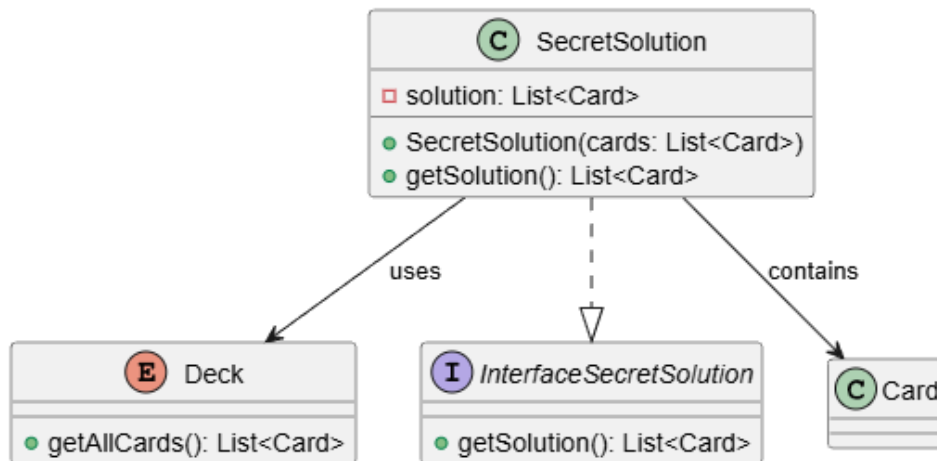
Gerarchia delle carte



Problema Il gioco prevede tre tipi di carte — personaggio, arma e stanza — che condividono attributi comuni come il nome e il tipo, ma che devono poter essere distinte e trattate in modo specifico dal resto del sistema.

Soluzione Si è scelto di modellare le carte tramite una classe astratta *Card* piuttosto che tramite una semplice interfaccia. Questa scelta è motivata dal fatto che tutte le carte condividono un'implementazione comune del campo *name* e del metodo *getName()*: utilizzare un'interfaccia avrebbe costretto a duplicare tale logica in ciascuna sottoclasse, violando il principio DRY (Don't Repeat Yourself). La classe astratta permette invece di centralizzare il comportamento comune, delegando alle sottoclassi solo la specializzazione necessaria. Le tre classi concrete *Characters*, *Weapons* e *Rooms* estendono *Card* e implementano ciascuna il metodo astratto *getType()*, restituendo il rispettivo valore dell'enumerazione *CardType*.

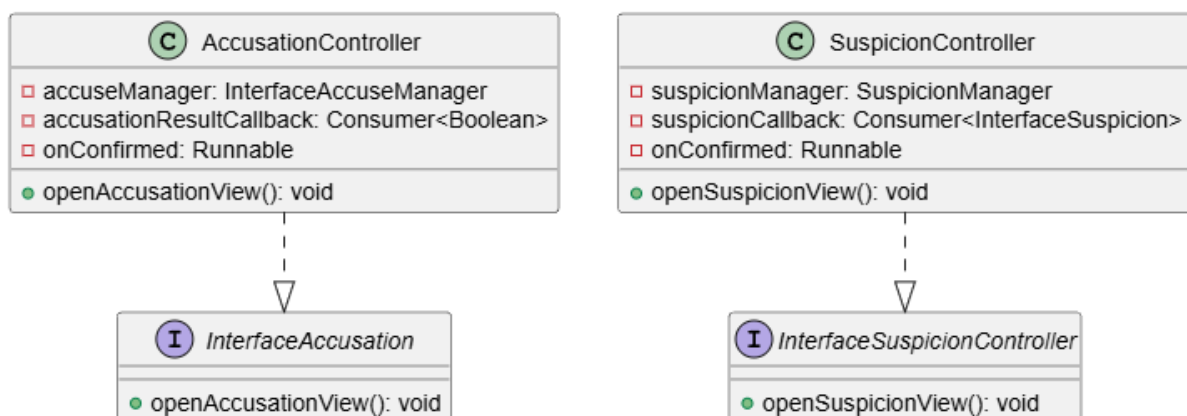
Generazione della soluzione segreta



Problema La soluzione segreta deve essere composta da esattamente una carta per ogni tipo (personaggio, arma e stanza), scelta casualmente dal mazzo all'inizio della partita, replicando fedelmente il comportamento del gioco da tavolo originale. Le carte estratte non devono inoltre essere distribuite ai giocatori.

Soluzione Il mazzo completo è modellato tramite l'enum *Deck*, che raccoglie tutte le 21 carte del gioco suddivise per tipo. Al momento della costruzione di *SecretSolution*, il mazzo viene mescolato tramite *Collections.shuffle()*, garantendo casualità ad ogni partita. Si scorrono poi le carte nell'ordine casuale ottenuto e si seleziona la prima carta incontrata per ciascun tipo. Le tre carte selezionate vengono infine rimosse dal mazzo originale, in modo che non possano essere distribuite ai giocatori durante la fase di setup, replicando fedelmente la meccanica del gioco fisico.

Disaccoppiamento tra controller e flusso di gioco

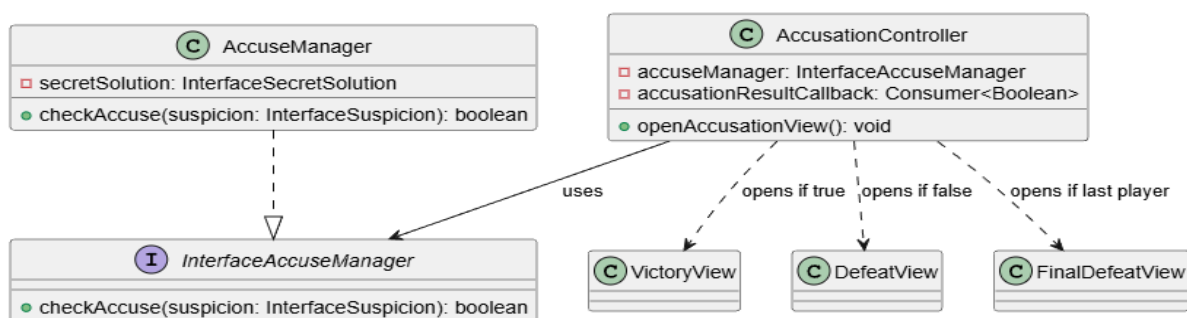


Problema I controller delle ipotesi e delle accuse devono comunicare l'esito delle operazioni al resto del gioco — ad esempio disabilitare i pulsanti d'azione o aggiornare lo stato della partita — senza essere accoppiati direttamente alle componenti che ricevono tali notifiche.

Soluzione Si è scelto di adottare un meccanismo di callback tramite le interfacce funzionali *Runnable* e *Consumer*, entrambe del package *java.util.function*, passate al costruttore dei controller *AccusationController* e *SuspicionController*. Il *Runnable onConfirmed* viene invocato immediatamente alla conferma da parte del giocatore, per disabilitare i pulsanti d'azione nella view di gioco, permettendone la logica corretta, senza che il controller sappia cosa accade concretamente. Il *Consumer<Boolean>* in *AccusationController* riceve il risultato booleano della verifica dell'accusa e lo propaga al chiamante, che deciderà quale view aprire.

Il *Consumer<InterfaceSuspicion>* in *SuspicionController* riceve invece l'ipotesi appena creata e la consegna al resto del sistema per la gestione successiva. In questo modo i controller non conoscono l'identità del ricevente e si limitano a invocare le callback con il risultato prodotto, restando completamente disaccoppiati dal flusso di gioco e rispettando il pattern MVC.

Verifica dell'accusa e aggiornamento della view



Problema Quando un giocatore formula un'accusa, il risultato deve essere verificato comparandolo alla soluzione segreta e la view deve aggiornarsi di conseguenza, mostrando la schermata di vittoria o di sconfitta. È necessario che questa logica di verifica non sia conosciuta né gestita dalla view stessa.

Soluzione La verifica dell'accusa è interamente delegata al model tramite *AccuseManager.checkAccuse()*, che confronta le tre carte selezionate dal giocatore con la soluzione segreta e restituisce un valore booleano. Il controller riceve tale risultato tramite un *Consumer<Boolean>* e lo propaga al resto del sistema, che provvede ad aprire la view corretta — *VictoryView* in caso di accusa corretta, *DefeatView* in caso contrario (*FinalDefeatView* nel caso in cui fosse l'ultimo giocatore a fare l'accusa). In questo modo la view rimane completamente ignara della logica di verifica, e il model non dipende in alcun modo dalla rappresentazione grafica, rispettando rigorosamente la separazione dei ruoli prevista dal pattern MVC.

3. Sviluppo

3.1 Testing automatizzato

Il progetto CluedoLite include dei test automatizzati sviluppati con JUnit, con l'obiettivo di garantire la correttezza della logica di gioco e la stabilità delle funzionalità principali. Poiché il gioco presenta numerose interazioni tra componenti, come la gestione dei turni, la formulazione dei sospetti, la generazione delle carte e la validazione delle accuse, è stato necessario strutturare i test in modo modulare, isolando le varie parti del model e verificandone il comportamento in maniera indipendente.

La maggior parte dei test si concentra sui pacchetti del **model**, in particolare su **gameflow**, **turnmanager**, **accuseandsuspect**, **gameboard**, **suspectnotes** e **player**. Questi componenti rappresentano il nucleo logico del gioco e sono completamente indipendenti dalla view, rendendoli ideali per il testing automatizzato. Per ogni modulo sono stati creati test specifici che verificano sia i casi standard sia quelli limite, come ad esempio la gestione di sospetti non validi, la corretta rotazione dei turni, la generazione coerente della soluzione segreta e la validazione delle accuse.

Un'attenzione particolare è stata dedicata al **TurnManager**, che coordina l'avanzamento della partita. Sono stati testati scenari come: cambio del giocatore attivo, impossibilità di formulare sospetti fuori da una stanza, gestione della fine del turno e sincronizzazione con il GameFlow. Per garantire la correttezza del comportamento, sono stati simulati turni completi con sequenze di azioni tipiche del gioco.

Il pacchetto **gameboard** è stato testato verificando la corretta gestione delle stanze, delle adiacenze e della posizione dei giocatori. I test includono la verifica che `getRooms()` restituisca esattamente nove stanze e una copia immutabile, che `getRoomByName()` trovi correttamente una stanza sia con maiuscole che minuscole e restituisca null per nomi inesistenti, e che `getAdjacent()` restituisca anch'essa una copia immutabile. Sono inoltre verificate la circolarità delle adiacenze tra la prima e l'ultima stanza, la presenza di adiacenza tra stanze consecutive e l'assenza di adiacenza tra stanze non consecutive. Il tracciamento della posizione dei giocatori è testato tramite `setPlayerPosition()` e `getPlayerPosition()`, verificando anche il caso in cui un giocatore non sia ancora stato posizionato.

Il test di **GameBoardControllerImpl** verifica il primo posizionamento del giocatore: dopo una chiamata a `move()`, la posizione del giocatore corrente nel model viene

aggiornata correttamente alla stanza indicata e la view viene notificata tramite `repaint()`.

Il pacchetto **gamesetup** viene testato in due modi:

- **secretSolution**, cioè la generazione della soluzione segreta è stata verificata tramite test dedicati, che controllano che la soluzione contenga esattamente tre carte, una per ciascuna categoria (personaggio, arma e stanza). I test verificano inoltre che le carte selezionate vengano rimosse correttamente dal mazzo e che il numero totale di carte rimanenti sia coerente (18 carte dopo la rimozione delle tre carte della soluzione). Questo garantisce l'assenza di duplicati e il rispetto dei vincoli del gioco.
- **cardDistribution**, cioè la distribuzione delle carte ai giocatori è stata testata separatamente, verificando che le 18 carte rimanenti dopo la creazione della soluzione segreta vengano distribuite in modo equo tra i giocatori. I test coprono diversi numeri di partecipanti e controllano che ogni giocatore riceva un numero corretto di carte, rispettando la suddivisione prevista dal regolamento (ad esempio 6 carte ciascuno con 3 giocatori, oppure distribuzioni 4–5 carte con 4 giocatori). Questo assicura che la distribuzione sia bilanciata e priva di incongruenze.

Il modulo **suspectnotes**, responsabile della gestione della tabella dei sospetti, è stato testato verificando la corretta creazione della tabella, la classificazione delle carte per categoria e l'aggiornamento dello stato delle caselle. I test controllano che le carte possedute dal giocatore vengano automaticamente escluse e che i metodi di aggiornamento (`excludeCard`, `updateTable`, `alreadyExcluded`) mantengano uno stato coerente.

I moduli **gameflow** e **game** sono stati sottoposti a test approfonditi per garantire la correttezza dell'intero ciclo di vita della partita. I test verificano le transizioni di stato (MENU → WAITING → IN_PROGRESS), la gestione dei giocatori in fase di lobby, l'assegnazione dei personaggi e il comportamento delle operazioni di reset e ritorno al menu. Sono inoltre controllati la validità del numero di giocatori, l'inserimento dei partecipanti, la coerenza della lista dei personaggi disponibili e la prevenzione di assegnazioni duplicate.

L'approccio adottato ha permesso di ottenere una copertura elevata delle funzionalità critiche del gioco, riducendo il rischio di regressioni.

3.2 Note di sviluppo

3.2.1 Giulia Abbondanza

- Utilizzo di Stream, lambda expression e method reference

Classe: TableImpl

Permalink:

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/model/suspectnotes/impl/TableImpl.java#L45>

Permalink:

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/model/suspectnotes/impl/TableImpl.java#L64>

Permalink:

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/model/suspectnotes/impl/TableImpl.java#L74>

- Utilizzo di List.copyOf() per restituzione immutabile

Classe: TableImpl

Permalink:

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/model/suspectnotes/impl/TableImpl.java#L56>

- Utilizzo di computeIfAbsent() su Map per lazy initialization

Classe: TableControllerImpl

Permalink:

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/controller/tablecontroller/impl/TableControllerImpl.java#L60>

- Ridefinizione di paintComponent() per rendering personalizzato

Classe: CardPanel

Permalink:

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/view/tableview/CardPanel.java#L90>

3.2.2 Giorgia Pivi

- Utilizzo di Stream con filter() e findFirst()

Classe: PlayerImpl

Permalink:

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/model/player/impl/PlayerImpl.java#L86>

- Utilizzo di Optional

Classe: PlayerImpl

Permalink:

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/model/player/impl/PlayerImpl.java#L79>

- Utilizzo di List.copyOf() e Collections.unmodifiableList()

Classe: PlayerImpl, GameImpl

Permalink:

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/model/player/impl/PlayerImpl.java#L72>

Permalink:

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/model/gameflow/impl/GameImpl.java#L67>

Permalink:

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/model/gameflow/impl/GameImpl.java#L62>

- Utilizzo di List.of()

Classe: GameImpl

Permalink:

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/model/gameflow/impl/GameImpl.java#LXX>

- Utilizzo di replaceAll() con lambda

Classe: GameImpl

Permalink:

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/model/gameflow/impl/GameImpl.java#L27>

- Utilizzo di Supplier<JFrame>
Classe: QuitButtonControllerImpl
Permalink:
<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/controller/buttonflowcontroller/impl/QuitButtonControllerImpl.java#L26>

3.2.3 Gessica Raimondi

- Utilizzo di Stream

Classe: GameBoardModelImpl
Permalink:
<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/0c1f66d529dbf41b92ab9d4a145c517b4c139367/src/main/java/it/unibo/cluedolite/model/gameboard/impl/GameBoardModelImpl.java#L53>
- Utilizzo di Optional

Classe: TurnManagerImpl
Permalink:
<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/0c1f66d529dbf41b92ab9d4a145c517b4c139367/src/main/java/it/unibo/cluedolite/model/turnmanager/impl/TurnManagerImpl.java#L79>
- Utilizzo di lambda expression e forEach

Classe: GameBoardModelImpl
Permalink:
<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/0c1f66d529dbf41b92ab9d4a145c517b4c139367/src/main/java/it/unibo/cluedolite/model/gameboard/impl/GameBoardModelImpl.java#L32>
- Utilizzo di List.copyOf()

Classe: RoomImpl

Permalink:

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/0c1f66d529dbf41b92ab9d4a145c517b4c139367/src/main/java/it/unibo/cluedolite/model/gameboard/impl/RoomImpl.java#L40>

Classe: GameBoardModelImpl

Permalink:

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/c2c6660098eff725123b266ebb7d6ff202efea1f/src/main/java/it/unibo/cluedolite/model/gameboard/impl/GameBoardModelImpl.java#L45>

- Utilizzo di @FunctionalInterface

Classe: EndTurnController

Permalink:

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/0c1f66d529dbf41b92ab9d4a145c517b4c139367/src/main/java/it/unibo/cluedolite/controller/endturnbuttoncontroller/api/EndTurnController.java#L6>

3.2.4 Valentina Severi

- Utilizzo di IntStream e lambda expression

Classe: CardDistribution

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/model/gamesetup/impl/CardDistribution.java#L22>

- Utilizzo di Stream, lambda expression e method reference

Classe: Deck

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/model/gamesetup/impl/Deck.java#L90>

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/model/gamesetup/impl/Deck.java#L102>

- Utilizzo di Consumer<Boolean> e Consumer<InterfaceSuspicion>

Classe: AccusationController

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/>

[java/it/unibo/cluedolite/controller/accuseandsuspectcontroller/impl/AccusationController.java#L34](https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/controller/accuseandsuspectcontroller/impl/AccusationController.java#L34)

Classe: SuspicionController

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/controller/accuseandsuspectcontroller/impl/SuspicionController.java#L38>

- Utilizzo di Runnable come callback

Classe: AccusationController

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/controller/accuseandsuspectcontroller/impl/AccusationController.java#L35>

Classe: SuspicionController

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/controller/accuseandsuspectcontroller/impl/SuspicionController.java#L38>

- Utilizzo di Supplier<Card> e Supplier<Player>

Classe: SuspicionController

<https://github.com/gessicaraimondi2/OOP25-CluedoLite/blob/master/src/main/java/it/unibo/cluedolite/controller/accuseandsuspectcontroller/impl/SuspicionController.java#L37>

4. Commenti finali

4.1 Autovalutazione e lavori futuri

4.1.1 Giulia Abbondanza

Mi sono occupata della tabella dei sospetti, componente che permette a ciascun giocatore di tenere traccia delle carte già escluse durante la partita. Ho gestito la logica di stato di ogni carta, il suo aggiornamento al variare degli eventi di gioco, la gestione separata della tabella per ciascun giocatore al cambio di turno, e la visualizzazione grafica con sezioni collassabili e un blocco note personale.

Sono soddisfatta della soluzione adottata per mantenere separata la tabella di ogni giocatore: ogni volta che scatta il turno di un nuovo giocatore, la sua tabella viene creata solo se non esiste ancora, evitando lavoro inutile. Sono soddisfatta anche del modo in cui le carte escluse vengono rese visivamente riconoscibili, con un'icona grafica immediata senza bisogno di componenti aggiuntivi.

Avrei potuto migliorare il modo in cui la view viene aggiornata: attualmente il componente grafico conosce direttamente lo stato interno del model, mentre in un design più rigoroso l'aggiornamento sarebbe dovuto passare interamente attraverso il controller.

Nel gruppo mi sono occupata della mia parte in modo autonomo, collaborando con le colleghe durante le fasi di integrazione e risoluzione dei conflitti di merge.

4.1.2 Giorgia Pivi

La parte di cui mi sono occupata riguarda la gestione del flusso iniziale della partita e il controllo dello stato globale del gioco. In particolare ho progettato e implementato l'enumerazione GameState e la logica di transizione all'interno di GameImpl, i controller del menu (StartControllerImpl e LobbyControllerImpl), la gestione dello stato dei giocatori tramite PlayerImpl e i controller delle azioni critiche QuitButtonControllerImpl e ResetButtonControllerImpl.

Sono soddisfatta soprattutto di due aspetti. Il primo è la gestione degli stati tramite GameState: modellare esplicitamente il ciclo di vita della partita ha reso il codice resistente a errori, impedendo transizioni non valide tramite IllegalStateException. Il secondo è la validazione dell'unicità dei personaggi in LobbyControllerImpl: centralizzare questo controllo nel controller ha mantenuto il model pulito e la view isolata dalla logica di business, rispettando il pattern MVC.

Se dovessi rivedere il mio lavoro, migliorerei il disaccoppiamento in QuitButtonControllerImpl. Al momento il controller istanzia direttamente new StartView(new StartControllerImpl()) dopo la conferma del quit, creando un accoppiamento esplicito con la view successiva. Una soluzione più pulita sarebbe passare un Runnable come callback nel costruttore: quando l'utente conferma, il controller si limiterebbe a invocarlo, senza sapere cosa accade concretamente. Questo renderebbe il controller completamente ignaro della presentazione, coerentemente con il pattern MVC.

4.1.3 Gessica Raimondi

La parte di cui mi sono occupata riguarda la gestione del tabellone di gioco: la logica di movimento dei giocatori(*GameBoardModelImpl* e *GameBoardControllerImpl*) e il coordinamento dei turni(*TurnManagerImpl*, *GameBoardControllerImpl* e *EndTurnControllerImpl*). Più nello specifico, ho implementato le regole che determinano come e quando un giocatore può spostarsi, i cambi di turno e la risoluzione circolare dei sospetti tramite il *TurnManager*. Ho gestito il collegamento tra controller e model, garantendo che ogni azione seguisse correttamente il flusso del gioco e mantenendo una chiara distinzione delle responsabilità tra i vari componenti.

Sono soddisfatta soprattutto della separazione raggiunta tra model e controller nella logica di movimento: il model mantiene le adiacenze fra le stanze e le posizioni dei giocatori, mentre il controller introduce i vincoli specifici del turno. Anche il disaccoppiamento tra controller e view si è rivelato efficace: il controller interagisce solo tramite l'interfaccia dedicata, evitando legami con l'implementazione grafica concreta e mantenendo il tabellone facilmente estendibile.

Se dovessi rivedere il mio lavoro, renderei il metodo *move()* in *GameBoardControllerImpl* più leggibile estraendo dei metodi privati dedicati. Attualmente il metodo gestisce più casi diversi all'interno dello stesso corpo: il primo posizionamento, il vincolo di *turnStartRoom* e il blocco del movimento. Estrarre metodi come *isFirstPlacement()* o *isValidMove()* non cambierebbe la logica ma renderebbe il codice più chiaro e più semplice da mantenere in futuro.

4.1.4 Valentina Severi

Il mio contributo principale ha riguardato due macro-aree del progetto: il setup della partita, che comprende la creazione del mazzo, la generazione della soluzione segreta e la distribuzione delle carte ai giocatori, e l'intera fase di sospetto e accusa, dalla formulazione alla verifica della correttezza dell'accusa, fino alle interfacce grafiche collegate.

Dal punto di vista progettuale sono soddisfatta della gerarchia delle carte, modellata tramite una classe astratta *Card* con sottoclassi concrete per ciascuna categoria, che ha evitato duplicazioni. Sono soddisfatta anche della distribuzione delle carte, realizzata in modo pulito tramite *Stream*, e della gestione della verifica dell'accusa, dove l'uso di *Consumer* e *Runnable* ha permesso di mantenere il controller disaccoppiato dal resto

del flusso di gioco. Sono invece meno soddisfatta della generazione della soluzione segreta: l'implementazione attuale mescola l'intero mazzo e seleziona la prima carta di ogni categoria, il che funziona correttamente ma delega tutta la casualità a un singolo *shuffle* globale. Un approccio più robusto sarebbe stato suddividere preventivamente le carte per categoria e applicare *Random.nextInt()* su ciascuna sottolista in modo indipendente, rendendo la casualità esplicita e uniforme per ogni categoria.

4.2 Difficoltà incontrate e commenti per i docenti

4.2.1 Giulia Abbondanza

La difficoltà principale ha riguardato la gestione della tabella dei sospetti in ottica multi-giocatore: garantire che ogni giocatore avesse il proprio stato isolato, mantenuto per l'intera partita e aggiornato correttamente al cambio di turno, ha richiesto diversi tentativi prima di trovare la soluzione giusta.

Un'altra difficoltà è stata la realizzazione del rendering grafico personalizzato delle carte, in particolare la gestione del ciclo di ridisegno e la sovrapposizione di più elementi grafici senza artefatti visivi.

La collaborazione tramite Git ha presentato qualche difficoltà nelle fasi di integrazione, ma ha rappresentato un'esperienza formativa importante per comprendere le dinamiche di sviluppo in team.

4.2.2 Giorgia Pivi

La principale difficoltà che ho incontrato durante il progetto è stata comprendere come strutturare correttamente l'interazione tra model, view e controller mantenendo un buon livello di disaccoppiamento tra le componenti. Nelle prime fasi tendevo infatti a creare collegamenti troppo diretti tra le diverse parti dell'applicazione; con l'avanzare dello sviluppo ho però acquisito una maggiore consapevolezza dell'architettura MVC e di come distribuire correttamente le responsabilità tra i vari livelli del sistema.

Un'altra sfida significativa è stata la gestione del lavoro di gruppo attraverso Git, in particolare durante l'integrazione delle funzionalità sviluppate dai diversi membri del team. Sebbene si siano verificati alcuni conflitti di merge, questa esperienza mi ha permesso di comprendere meglio l'importanza della collaborazione, della

comunicazione all'interno del gruppo e del mantenimento di una struttura del codice coerente e condivisa.

Nel complesso, queste difficoltà si sono trasformate in occasioni di apprendimento che mi hanno aiutata a migliorare sia dal punto di vista tecnico sia nella gestione del lavoro in team.

4.2.3 Gessica Raimondi

La difficoltà principale che ho incontrato ha riguardato la progettazione della logica con l'architettura MVC. All'inizio non era immediato capire come distribuire correttamente le responsabilità tra model e controller senza creare dipendenze indesiderate, soprattutto nella gestione dei vincoli del turno e delle condizioni che determinano se un giocatore può muoversi. Questa prima parte ha richiesto qualche tentativo, ma alla fine sono riuscita a capire questo tipo di architettura e a definire una struttura chiara per la gestione della logica.

Un'altra difficoltà significativa è stata l'unione delle diverse parti sviluppate dal gruppo come tabellone, sospetti, controller e interfacce grafiche; ha richiesto coordinazione e diversi aggiustamenti per garantire coerenza e corretto funzionamento dell'insieme. Nel complesso è stata un'esperienza che mi ha aiutato a comprendere più chiaramente le dinamiche del lavoro di squadra all'interno di progetti di questo tipo.

4.2.4 Valentina Severi

La principale difficoltà incontrata durante lo sviluppo ha riguardato la progettazione del meccanismo di callback per la gestione dei sospetti e delle accuse. Non era immediato capire come disaccoppiare i controller dal resto del flusso di gioco mantenendo il codice pulito e rispettando il pattern MVC.

Il progetto è stato comunque una sfida personale stimolante, soprattutto considerando che non avevo mai programmato in Java prima d'ora: la difficoltà maggiore non è stata tanto la scrittura del codice quanto la comprensione di come le varie parti del sistema dovessero integrarsi tra loro.

A Guida utente

Avvio del gioco

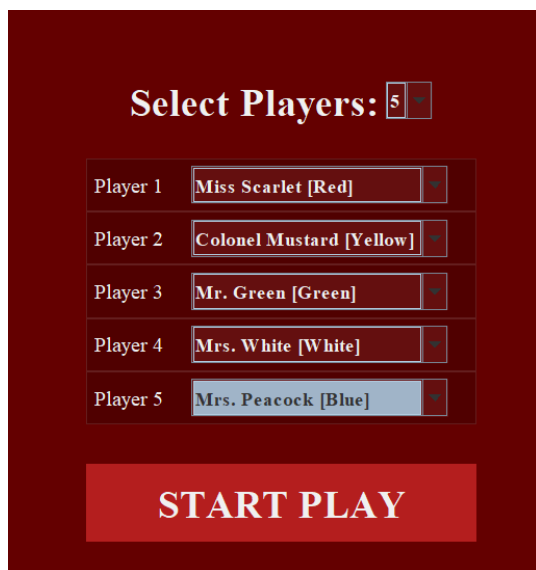
Il gioco può essere avviato in due modi:

- Doppio click sul file OOP25-CluedoLite-all.jar
- Da terminale con il comando: java -jar OOP25-CluedoLite-all.jar

Configurazione della partita



All'avvio viene mostrata la schermata del menu principale. Il gioco supporta da 3 a 6 giocatori. Ciascun giocatore deve selezionare un personaggio disponibile. Una volta che tutti i giocatori sono stati configurati, è possibile avviare la partita premendo il pulsante Start Game.

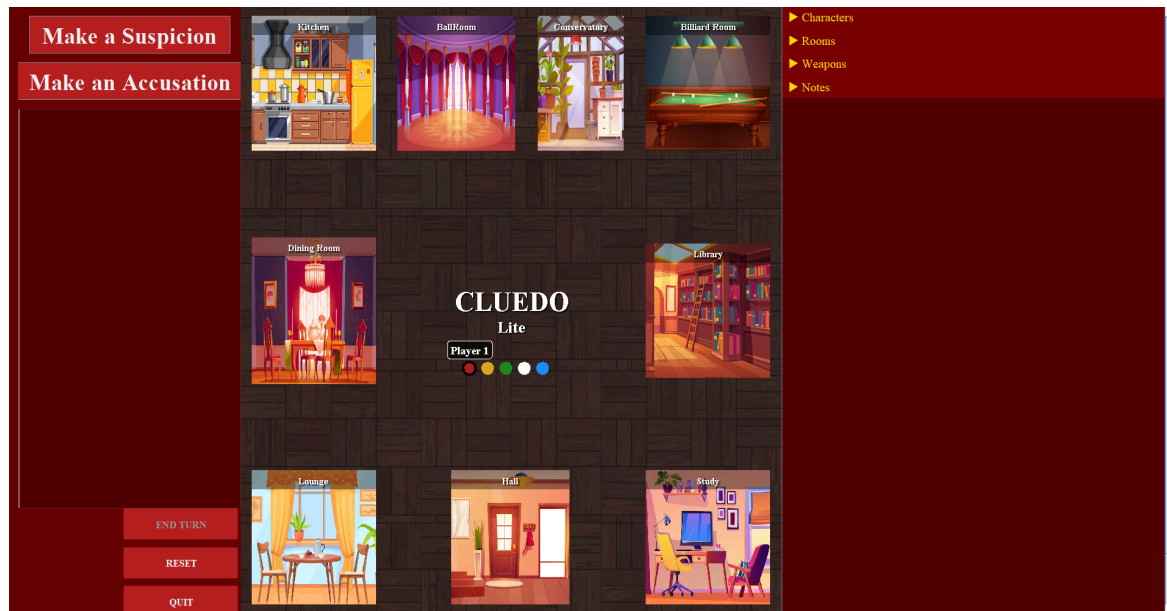


La partita si inizializza automaticamente: le carte vengono distribuite ai giocatori, la tabella dei sospetti viene creata e vengono già spuntate le carte della propria mano e la soluzione segreta (un personaggio, un'arma e una stanza) viene generata casualmente e nascosta.

Schermata di gioco

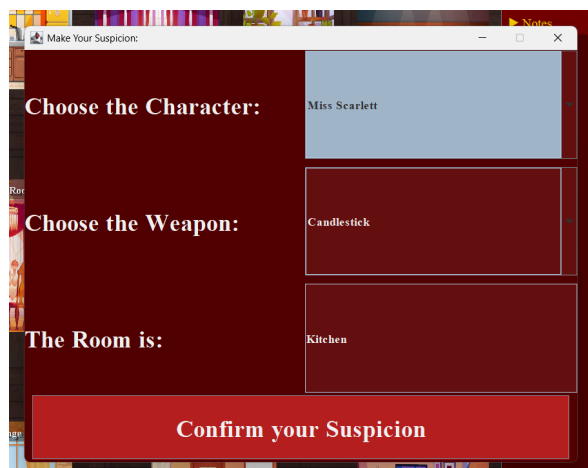
La schermata principale di gioco è divisa in tre sezioni:

- Sinistra: pannello dei pulsanti di azione
- Centro: tabellone di gioco con i personaggi
- Destra: tabella dei sospetti personale del giocatore corrente

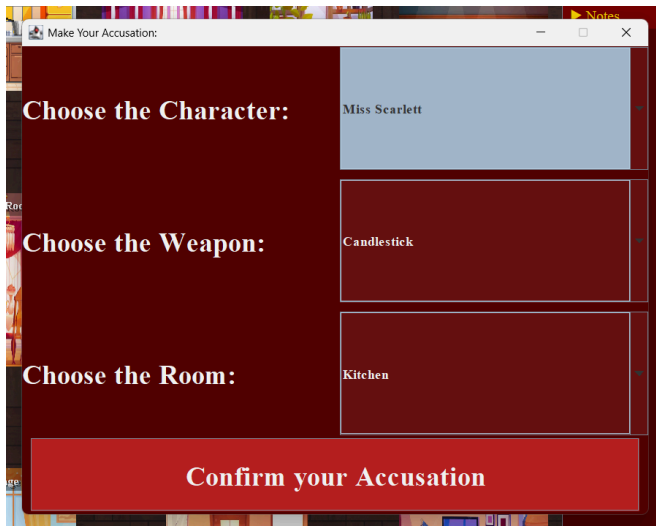


Pulsanti di azione

Make a Suspicion: apre una finestra in cui il giocatore può selezionare un personaggio, un'arma e la stanza in cui si trova per avanzare un sospetto. Disponibile solo se il giocatore si trova in una stanza.



Make an Accusation: apre una finestra in cui il giocatore può selezionare personaggio, arma e stanza per fare un'accusa formale. Se l'accusa è corretta il giocatore vince; se è sbagliata viene eliminato.



End Turn: termina il turno del giocatore corrente, passa al successivo e chiude automaticamente la tabella in modo tale che non venga visualizzata da altri giocatori.

Reset: riavvia la partita dall'inizio con gli stessi giocatori, previa conferma.

Quit: torna al menu principale, previa conferma.

Flusso di un turno

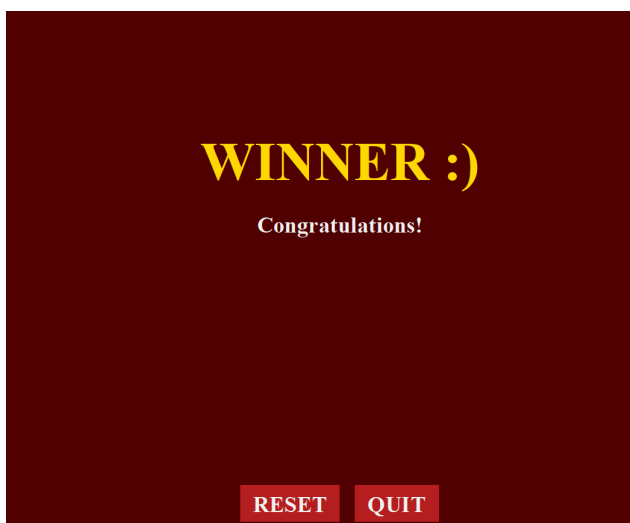
Il giocatore muove il proprio personaggio sul tabellone.

Se si trova in una stanza, può avanzare un sospetto o fare un'accusa.

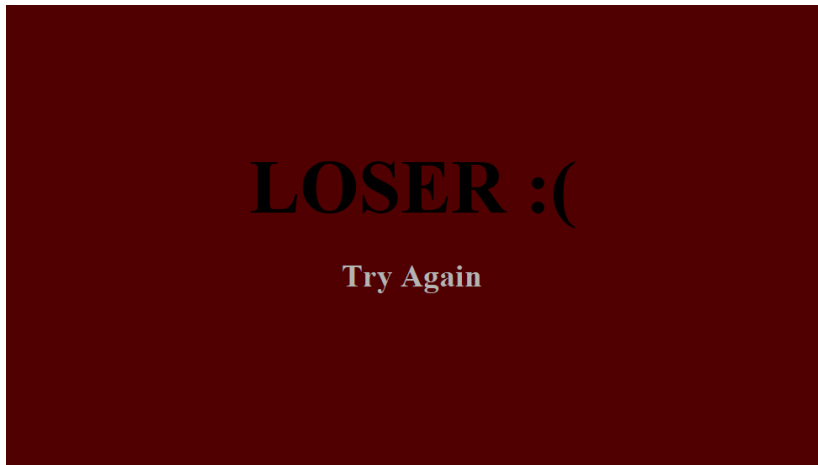
Termina il turno premendo End Turn.

Vittoria e sconfitta

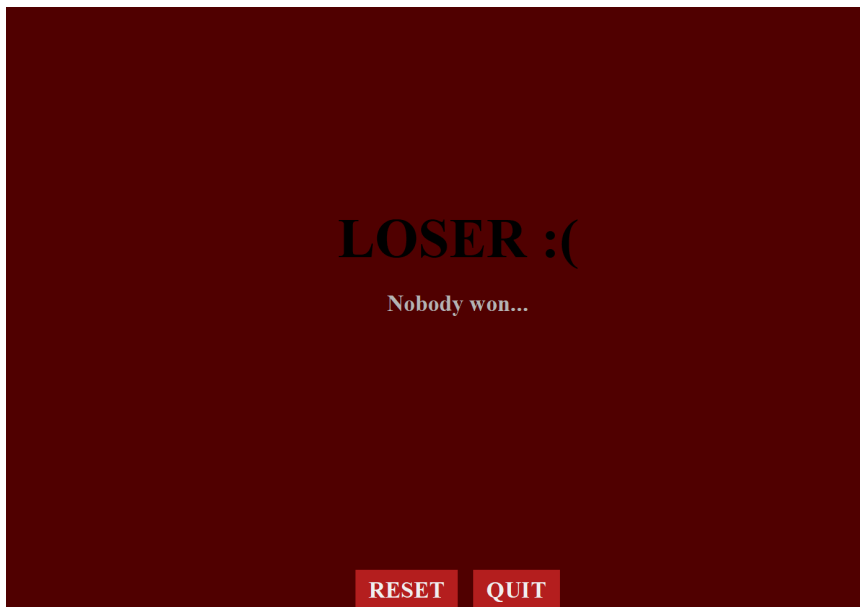
Vittoria: se un giocatore fa un'accusa corretta, viene mostrata la schermata di vittoria.



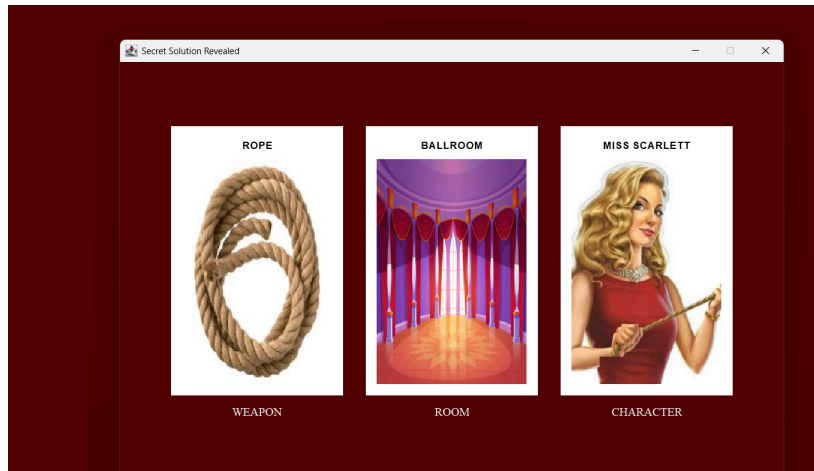
Sconfitta temporanea: se un giocatore fa un'accusa sbagliata viene eliminato e non potrà più fare sospetti né accuse, ma la partita continua con gli altri giocatori. Viene mostrata brevemente una schermata di sconfitta.



Sconfitta finale: se tutti i giocatori tranne uno vengono eliminati, l'ultimo rimasto è costretto a fare un'accusa finale. Se anche questa è sbagliata, nessuno ha vinto e viene mostrata la schermata di sconfitta finale.



Dalla schermata di vittoria e da quella di sconfitta finale è possibile innanzitutto visualizzare la soluzione segreta



Inoltre si hanno due scelte:

- Reset: riavviare una nuova partita.
- Quit: tornare al menu principale.